2.4 Evaluation

For few-shot learning, we evaluate each example in the evaluation set by randomly drawing K examples from that task's training set as conditioning, delimited by 1 or 2 newlines depending on the task. For LAMBADA and Storycloze there is no supervised training set available so we draw conditioning examples from the development set and evaluate on the test set. For Winograd (the original, not SuperGLUE version) there is only one dataset, so we draw conditioning examples directly from it.

K can be any value from 0 to the maximum amount allowed by the model's context window, which is $n_{\text{ctx}} = 2048$ for all models and typically fits 10 to 100 examples. Larger values of K are usually but not always better, so when a separate development and test set are available, we experiment with a few values of K on the development set and then run the best value on the test set. For some tasks (see Appendix G) we also use a natural language prompt in addition to (or for $K = 0$, instead of) demonstrations.

On tasks that involve choosing one correct completion from several options (multiple choice), we provide K examples of context plus correct completion, followed by one example of context only, and compare the LM likelihood of each completion. For most tasks we compare the per-token likelihood (to normalize for length), however on a small number of datasets (ARC, OpenBookQA, and RACE) we gain additional benefit as measured on the development set by normalizing by the unconditional probability of each completion, by computing $\frac{P(\text{completion}|\text{context})}{P(\text{completion}|\text{answer_context})}$, where answer_context is the string `"Answer: "` or `"A: "` and is used to prompt that the completion should be an answer but is otherwise generic.

On tasks that involve binary classification, we give the options more semantically meaningful names (e.g. "True" or "False" rather than 0 or 1) and then treat the task like multiple choice; we also sometimes frame the task similar to what is done by [RSR+19] (see Appendix G) for details.

On tasks with free-form completion, we use beam search with the same parameters as [RSR+19]: a beam width of 4 and a length penalty of $\alpha = 0.6$. We score the model using F1 similarity score, BLEU, or exact match, depending on what is standard for the dataset at hand.

Final results are reported on the test set when publicly available, for each model size and learning setting (zero-, one-, and few-shot). When the test set is private, our model is often too large to fit on the test server, so we report results on the development set. We do submit to the test server on a small number of datasets (SuperGLUE, TriviaQA, PiQa) where we were able to make submission work, and we submit only the 200B few-shot results, and report development set results for everything else.

3 Results

In Figure 3.1 we display training curves for the 8 models described in Section 2. For this graph we also include 6 additional extra-small models with as few as 100,000 parameters. As observed in [KMH+20], language modeling performance follows a power-law when making efficient use of training compute. After extending this trend by two more orders of magnitude, we observe only a slight (if any) departure from the power-law. One might worry that these improvements in cross-entropy loss come only from modeling spurious details of our training corpus. However, we will see in the following sections that improvements in cross-entropy loss lead to consistent performance gains across a broad spectrum of natural language tasks.

Below, we evaluate the 8 models described in Section 2 (the 175 billion parameter parameter GPT-3 and 7 smaller models) on a wide range of datasets. We group the datasets into 9 categories representing roughly similar tasks.

In Section 3.1 we evaluate on traditional language modeling tasks and tasks that are similar to language modeling, such as Cloze tasks and sentence/paragraph completion tasks. In Section 3.2 we evaluate on "closed book" question answering tasks: tasks which require using the information stored in the model's parameters to answer general knowledge questions. In Section 3.3 we evaluate the model's ability to translate between languages (especially one-shot and few-shot). In Section 3.4 we evaluate the model's performance on Winograd Schema-like tasks. In Section 3.5 we evaluate on datasets that involve commonsense reasoning or question answering. In Section 3.6 we evaluate on reading comprehension tasks, in Section 3.7 we evaluate on the SuperGLUE benchmark suite, and in 3.8 we briefly explore NLI. Finally, in Section 3.9, we invent some additional tasks designed especially to probe in-context learning abilities – these tasks focus on on-the-fly reasoning, adaptation skills, or open-ended text synthesis. We evaluate all tasks in the few-shot, one-shot, and zero-shot settings.

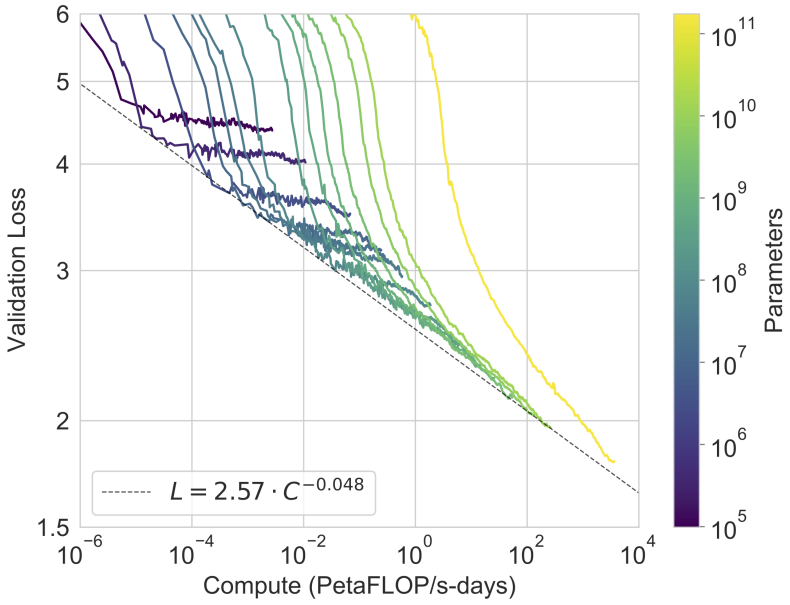

Figure 3.1: Smooth scaling of performance with compute. Performance (measured in terms of cross-entropy validation loss) follows a power-law trend with the amount of compute used for training. The power-law behavior observed in [KMH+20] continues for an additional two orders of magnitude with only small deviations from the predicted curve. For this figure, we exclude embedding parameters from compute and parameter counts.

Setting	PTB
SOTA (Zero-Shot)	35.8[a]
GPT-3 Zero-Shot	**20.5**

Table 3.1: Zero-shot results on PTB language modeling dataset. Many other common language modeling datasets are omitted because they are derived from Wikipedia or other sources which are included in GPT-3's training data. [a][RWC+19]

3.1 Language Modeling, Cloze, and Completion Tasks

In this section we test GPT-3's performance on the traditional task of language modeling, as well as related tasks that involve predicting a single word of interest, completing a sentence or paragraph, or choosing between possible completions of a piece of text.

3.1.1 Language Modeling

We calculate zero-shot perplexity on the Penn Tree Bank (PTB) [MKM+94] dataset measured in [RWC+19]. We omit the 4 Wikipedia-related tasks in that work because they are entirely contained in our training data, and we also omit the one-billion word benchmark due to a high fraction of the dataset being contained in our training set. PTB escapes these issues due to predating the modern internet. Our largest model sets a new SOTA on PTB by a substantial margin of 15 points, achieving a perplexity of 20.50. Note that since PTB is a traditional language modeling dataset it does not have a clear separation of examples to define one-shot or few-shot evaluation around, so we measure only zero-shot.

3.1.2 LAMBADA

The LAMBADA dataset [PKL+16] tests the modeling of long-range dependencies in text – the model is asked to predict the last word of sentences which require reading a paragraph of context. It has recently been suggested that the continued scaling of language models is yielding diminishing returns on this difficult benchmark. [BHT+20] reflect on the small 1.5% improvement achieved by a doubling of model size between two recent state of the art results ([SPP+19]

Setting	LAMBADA (acc)	LAMBADA (ppl)	StoryCloze (acc)	HellaSwag (acc)
SOTA	68.0[a]	8.63[b]	**91.8**[c]	**85.6**[d]
GPT-3 Zero-Shot	**76.2**	**3.00**	83.2	78.9
GPT-3 One-Shot	**72.5**	**3.35**	84.7	78.1
GPT-3 Few-Shot	**86.4**	**1.92**	87.7	79.3

Table 3.2: Performance on cloze and completion tasks. GPT-3 significantly improves SOTA on LAMBADA while achieving respectable performance on two difficult completion prediction datasets. [a][Tur20] [b][RWC+19] [c][LDL19] [d][LCH+20]

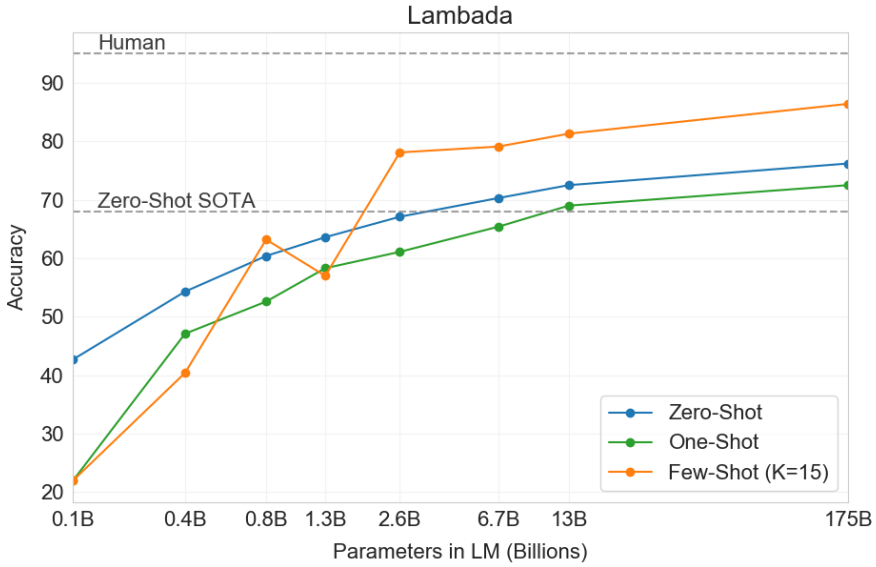

Figure 3.2: On LAMBADA, the few-shot capability of language models results in a strong boost to accuracy. GPT-3 2.7B outperforms the SOTA 17B parameter Turing-NLG [Tur20] in this setting, and GPT-3 175B advances the state of the art by 18%. Note zero-shot uses a different format from one-shot and few-shot as described in the text.

and [Tur20]) and argue that "continuing to expand hardware and data sizes by orders of magnitude is not the path forward". We find that path is still promising and in a zero-shot setting GPT-3 achieves 76% on LAMBADA, a gain of 8% over the previous state of the art.

LAMBADA is also a demonstration of the flexibility of few-shot learning as it provides a way to address a problem that classically occurs with this dataset. Although the completion in LAMBADA is always the last word in a sentence, a standard language model has no way of knowing this detail. It thus assigns probability not only to the correct ending but also to other valid continuations of the paragraph. This problem has been partially addressed in the past with stop-word filters [RWC+19] (which ban "continuation" words). The few-shot setting instead allows us to "frame" the task as a cloze-test and allows the language model to infer from examples that a completion of exactly one word is desired. We use the following fill-in-the-blank format:

Alice was friends with Bob. Alice went to visit her friend _____. → Bob

George bought some baseball equipment, a ball, a glove, and a _____. →

When presented with examples formatted this way, GPT-3 achieves 86.4% accuracy in the few-shot setting, an increase of over 18% from the previous state-of-the-art. We observe that few-shot performance improves strongly with model size. While this setting decreases the performance of the smallest model by almost 20%, for GPT-3 it improves accuracy by 10%. Finally, the fill-in-blank method is not effective one-shot, where it always performs worse than the zero-shot setting. Perhaps this is because all models still require several examples to recognize the pattern.